E7: Introduction to Computer Programming
Midterm 1A

Monday, September 30

Full Name

Student ID Number

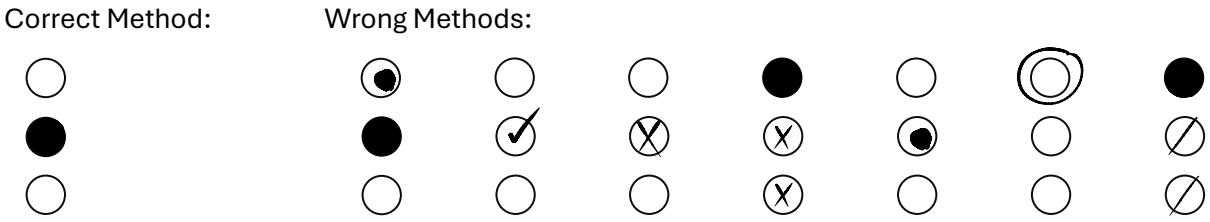
Instructions:

- You have 1 hour and 20 minutes to complete this exam.
- The exam is closed book, closed notes, closed computer, closed calculator.
- You may only use one 8.5" × 11" sheet of notes (both sides). No other resources are permitted.
- Mark your answers on the exam paper within the designated spaces. Responses written on scratch paper or outside of the specified answer spaces will not be graded.
- This exam is DOUBLE-SIDED. Make sure to read both sides of each page.
- The number of points associated with each part of each question is indicated.
- The exam is worth a total of 100 points.
- For questions with circular bubbles, you should select exactly *one* choice.
  - ◯ You may only chose one option!
  - ◯ Or this one, but not both!
  - ◯ Or this one, but no more than one!
- For questions with square boxes, you may select *multiple* choices.
  - ☐ You can select this one!
  - ☐ You can select this one too!
  - ☐ You can select one, two, or all three of these!
- Only include your FINAL answers in the designated boxes. Any additional text will count as part of your answer and may result in a reduced grade.

Good luck!

Question 1 (30 points)

There are 10 multiple choice questions. There is only **ONE** correct choice for each question. Multiple bubbles, incomplete bubbles, or stray marks will be marked incorrect.



1. (3 pt) How do you insert a comment in Python code?

- ● `# This is a comment`
- ○ `// This is a comment`
- ○ `% This is a comment`
- ○ `'This is a comment'`
- ○ `/* This is a comment */`
- ○ `"This is a comment"`

2. (3 pt) What is the output of the following code?

```
x = "ENGIN 7"
print(x[1])
```

- ○ `7`
- ○ `1`
- ● `N`
- ○ `n`
- ○ `Error`
- ○ `E`
- ○ `ENGIN 7`
- ○ `e`

3. (3 pt) What is the output of the following code?

```
x = 21
x + 3
x == 25
print(x)
```

- ○ `(x)`
- ○ `25`
- ○ `False`
- ● `21`
- ○ None of the options
- ○ `x`
- ○ `Error`
- ○ `24`
- ○ `True`

4. (3 pt) Which of the following is best suited to iterate over a data structure in Python?

- ○ `repeat-until` loop
- ○ `while` loop
- ● `for` loop
- ○ `do-while` loop
- ○ recursion
- ○ `if` loop

5. (3 pt) What is the output of the following code?

```
x = 5
y = x
x = 10
x = y
print(y)
```

- ○ Error
- ○ 10
- ● 5
- ○ 15
- ○ x
- ○ None of the options

6. (3 pt) What is the output of the following code?

```
data = [0, 1, 1]
print(data * 2)
```

- ○ [0, 3, 3]
- ○ [0, 2, 2]
- ○ [0, 1, 1]
- ○ [0, 1, 1, 2]
- ○ Error
- ● [0, 1, 1, 0, 1, 1]

7. (3 pt) Which of the following is the correct way to create a variable named `x` in Python?

- ○ x : 10
- ○ def x = 10
- ○ x == 10
- ○ int x = 10
- ● x = 10
- ○ var x = 10

8. (3 pt) What is the output of the following code?

```
x = '5'
y = "5"
print(x + y)
```

- ○ Error
- ○ 25
- ● 55
- ○ 5 + 5
- ○ (x + y)
- ○ 10

9. (3 pt) What is the output of the following code?

```
print(2 + 3 ** 2)
```

- ● 11
- ○ Error
- ○ '2 + 3 ** 2'
- ○ 8
- ○ 5
- ○ 25
- ○ 10
- ○ Nothing

10. (3 pt) How do you create a list named `seq` in Python?

- ○ seq = 1, 2, 3
- ● seq = [1, 2, 3]
- ○ seq = {1, 2, 3}
- ○ seq = "[1, 2, 3]"
- ○ seq = "(1, 2, 3)"
- ○ seq = (1, 2, 3)

Question 2 (25 points)

The function below computes the net force acting on a box that is being pulled horizontally, where `P` is the pulling force, `m` is the mass of the box, `g` is the gravitational acceleration, and `f` is the coefficient of friction. **For all parts, assume that the following function has been defined.**

```
def netForce(P, m, g=10, f=0):
    net =  P - m * g * f
    return net
```

What is the output of the following expressions? If an error occurs, write "Error". If nothing is displayed, write "Nothing". If the evaluation would run forever, write "Forever".

(a) (2 pt) `print(netForce(10, 1, 1))`

```
10
```

(b) (3 pt) `print(netForce(10, 1, f=1))`

```
0
```

(c) (3 pt) What is the output of the following code?

```
m = 10
P = 1
print(netForce(m, P))
```

```
10
```

(d) (3 pt) What is the output of the following code?

```
netForce(10, 1, 1, 1)
print(net)
```

```
Error
```

(e) (4 pt) Redefine the function `netForce()` using a single line of code. Your line of code should replace the function definition AND its body, all in a single line of code.

```
netForce = lambda P, m, g=10, f=0: P - m * g * f
```

Next, consider a box being pulled by an initial force `P` across multiple surfaces. If the net force (`P - m * g * f`) is greater than or equal to zero, the object moves to the next surface, where the coefficient of friction `f` gets doubled and the pulling force `P` is divided by 2. This process continues until the net force becomes negative. You need to determine the number of surfaces the object can cross before the net force becomes negative. Refer to the examples below. The code is shown on the next page.

Example 1: `P=1, m=1, g=1, f=1`

Surface 1: net force = `1 - 1 * 1 * 1 = 0` $\geq$ `0`

Object moves to the next surface with `P=0.5` and `f=2`

Surface 2: net force = `0.5 - 1 * 1 * 2 = -1.5` $<$ `0` (not enough to cross)

Return `1` (only crossed 1 surface)

Example 2: `P=0, m=1, g=1, f=1`

Surface 1: net force = `0 - 1 * 1 * 1 = -1` $<$ `0` (not enough to cross)

Return `0` (didn't cross any surfaces)

Example 3: `P=2, m=1, g=1, f=0.5`

Surface 1: net force = `2 - 1 * 1 * 0.5 = 1.5` $\geq$ `0`

Object moves to the next surface with `P=1` and `f=1`

Surface 2: net force = `1 - 1 * 1 * 1 = 0` $\geq$ `0`

Object moves to the next surface with `P=0.5` and `f=2`

Surface 3: net force = `0.5 - 1 * 1 * 2 = -1.5` $<$ `0` (not enough to cross)

Return `2` (crossed 2 surfaces)

Write a function `countSurfaces()` that takes scalar values `P, m, g, f`. The function should return the number of surfaces the box can cross before the net force becomes negative. You will implement the function by filling in the blanks. You may only write one statement on each line. You may not use ; to place two statements on the same line. You may not use any functions or methods, built-in or otherwise, other than `netForce()`. You may not modify the level of indentation of any line.

```
1 def countSurfaces(P, m, g, f):
2     """
3     Return number of surfaces before net force becomes negative
4     """
5     num_surfaces = 0
6     ______(f)______
7         num_surfaces = ______(g)______
8         P, f = ______(h)______
9     return num_surfaces
```

(f) (3 pt) Fill in blank (f).

```
while netForce(P, m, g, f) >= 0:
OR
while P-m*g*f >= 0:
```

(g) (2 pt) Fill in blank (g).

```
num_surfaces = num_surfaces + 1
```

(h) (3 pt) Fill in blank (h).

```
P, f = P/2, f*2
```

(i) (2 pt) Which of the following modifications will NOT affect the behavior of the `countSurfaces()` function? Select ALL that apply. Line numbers are indicated to the left.

- ☐ Increasing the indentation of line 9
- ☐ Deleting line 5
- ■ Swapping lines 7 and 8
- ☐ Changing line 5 to `num_surfaces = 1`
- ☐ None of the options
- ☐ Swapping lines 5 and 6
- ☐ Swapping lines 6 and 7
- ☐ Swapping lines 8 and 9

Question 3 (7 points)

You need to write a Python function `exists(n, sequence)` that takes a scalar value `n` and a 1D data structure `sequence`. The function should return `True` and the index of the first occurrence of `n` in `sequence` if `n` exists, or `False` otherwise. Refer to the examples below. Your friend from *Stanford* wrote the function below, but you notice it might contain errors. Your task is to fix it.

Any underlined words must remain unchanged.

```
1  def exists(n, sequence):
2      """Return whether n exists in sequence and the index of its first occurrence
3      >>> exists(4, [1, 2, 3, 4, 5])
4      (True, 3)
5      >>> exists(6, [1, 2, 3, 4, 5])
6      False
7      >>> exists(1, [])
8      False
9      """
10     for i in range(len(sequence) - 1):
11         if sequence[n] = i:
12             return False
13     return True
```

Line numbers are indicated to the left. Fix the function by rewriting the specified lines below. If a line is already correct, write "Correct" in the corresponding box; otherwise, write the fixed line of code. You may not use ; to place two statements on the same line. You may not use ANY functions or methods, built-in or otherwise, other than the ones already mentioned in the function above. Finally, you may not modify the level of indentation or any underlined words in the code above.

Line 1

```
Correct
```

Line 10

```
for i in range(len(sequence))
OR
for i in range(0, len(sequence))
OR
for i in range(0, len(sequence), 1)
```

Line 11

```
if sequence[i] == n:
```

Line 12

```
return True, i
```

Line 13

```
return False
```

Question 4 (13 points)

You need to write a Python function `countChanges(word)` which takes a string `word` and returns the number of character changes in the string. Assume that `word` only includes lowercase and/or uppercase letters. A character change occurs when one letter changes to another letter, such as `'ab'` or `'hi'`. However, changing between uppercase and lowercase of the same letter does not count as a character change, such as `'aA'` or `'Aa'`. Refer to the examples below for more details. If the input `word` is not of type string, the function should return the number `-999`.

Anything written on this page will NOT be graded.

```
def countChanges(word):
    """Return number of character changes in a string
    >>> countChanges('abbba') # 2 changes: a to b, and b to a
    2
    >>> countChanges('abBba') # 2 changes: a to b, and b to a
    2
    >>> countChanges('hello') # 3 changes: h to e, e to l, and l to o
    3
    >>> countChanges('eeEeeEeeEEee') # 0 changes
    0
    >>> countChanges(777) # input is not of type string
    -999
    """
    if ______(a)______:
        ______(b)______
    changes = ______(c)______
    for i in ______(d)______:
        if ______(e)______:
            changes += ______(f)______
    return changes
```

The following string methods might be helpful:

- `string.lower()`: Return a copy of the string with all characters converted to lowercase.
- `string.upper()`: Return a copy of the string with all characters converted to uppercase.
- `string.count(sub)`: Return the number of non-overlapping occurrences of `sub` in the string.
- `string.find(sub)`: Return the lowest index in the string where substring `sub` is found.

You will implement the function `countChanges()` by filling in the blanks. You may only write one statement on each line. You may not use ; to place two statements on the same line. Except when explicitly provided in multiple-choice questions, you may not use any functions or methods, built-in or otherwise. You may not modify the level of indentation of any line.

(a) (3 pt) Which of these could fill in blank (a)? Select ALL that apply.

- ■ `type(word) != str`
- ☐ `word == str`
- ■ `not type(word) == str`
- ☐ `isinstance(word, str)`
- ☐ `word != str`
- ☐ `word = str`
- ☐ `type(word) == str`
- ☐ `type(word) = str`
- ☐ `not type(word) = str`

(b) (2 pt) Fill in blank (b).

```
return -999
```

(c) (1 pt) Fill in blank (c).

```
changes = 0
```

(d) (2 pt) Which of these could fill in blank (d)?

- ○ `range(word)`
- ○ `len(word) - 1`
- ● `range(len(word) - 1)`
- ○ `word`
- ○ `length(word)`
- ○ `enumerate(word)`
- ○ `range(len(word))`
- ○ `len(word)`
- ○ `range(len(word) + 1)`

(e) (3 pt) Which of these could fill in blank (e)? Select ALL that apply.

- ☐ `not word[i] = word[i+1]`
- ☐ `word[i] != word[i-1]`
- ☐ `word[i] != word[i+1]`
- ☐ `i != i-1`
- ☐ `not i = i-1`
- ☐ `i != i+1`
- ☐ `not i = i+1`
- ☐ `word[i].lower() != word[i-1].lower()`
- ☐ `word[i].lower() ≠ word[i+1].lower()`
- ☐ `word[i].upper() ≠ word[i-1].upper()`
- ☐ `word[i].upper() != word[i-1].upper()`
- ☐ `word[i].upper() != word[i+1].lower()`
- ■ `word[i].upper() != word[i+1].upper()`
- ■ `word[i].lower() != word[i+1].lower()`

(f) (2 pt) Fill in blank (f).

```
changes += 1
```

Question 5 (7 points)

Each part is independent of the other.

(a) (4 pt) Given a data structure `sequence = [1, 2, 3, 4]`, which of the following code snippets will output only the value `2` when executed? Select ALL that apply.

■
```
for i in sequence:
    if i == 2:
        print(i)
```

□
```
for j in sequence:
    if j == 2:
        print(j)
    break
```

■
```
for q in sequence:
    if q % 2 == 0:
        print(q)
        break
```

■
```
for m in sequence:
    if m == 2:
        break
print(m)
```

■
```
for n in sequence:
    if n == 2:
        print(n)
        continue
```

□
```
for k in sequence:
    if k == 2:
        break
        print(k)
```

■
```
for p in sequence:
    if p != 2:
        continue
    print(p)
```

□
```
for o in sequence:
    if o == 2:
        continue
        print(o)
```

(b) (3 pt) Given `a = 4` and `b = 5`, which of the following code snippets will output only the value `4` when executed? Select ALL that apply.

■
```
if a < b:
    print(a)
elif a + b == 9:
    print(b)
```

□
```
if a < b:
    print(a)
print(b)
```

□
```
if a + b == 9 and a == b:
    if a < b:
        print(a)
```

■
```
if a < b:
    print(a)
else:
    print(b)
```

■
```
if a > b or a + b == 9:
    print(a)
else:
    print(b)
```

□
```
if a < b:
    print(a)
if a + b == 9:
    print(b)
```

Question 6 (8 points)

You obtained a sample of the presidential election polls. **For all parts below, assume that the following code snippet has been executed.**

```
import numpy as np
harris = np.array([50, 49, 51, 47, 53])
trump  = np.array([48, 47, 48, 49, 47])
```

What is the output of the following expressions? If an error occurs, write "Error". If nothing is displayed, write "Nothing". If the evaluation would run forever, write "Forever".

(a) (2 pt) `print(harris[:4:2])`

```
[50 51]
```

(b) (1 pt) `print(harris[-1])`

```
53
```

(c) (1 pt) `print(harris + 1)`

```
[51 50 52 48 54]
```

(d) (2 pt) `print(harris < 50)`

```
[False True False True False]
```

(e) (2 pt) What is the output of the following code?

```
result = [3 if trump[i] > harris[i] else 7 for i in range(len(harris) - 1)]
print(result)
```

```
[7, 7, 7, 3]
```

Question 7 (10 points)

Consider the following triangle of numbers and the sum of all numbers in that row:

```
row 1:          1            sum = 1
row 2:        2 3            sum = 2 + 3 = 5
row 3:      4 5 6            sum = 4 + 5 + 6 = 15
row 4:    7 8 9 10           sum = 7 + 8 + 9 + 10 = 34
...          ...             ...
```

First, you need to write a Python function `first(n)` which takes an integer `n` representing the row number (assume `n` ≥ 1). The function should use recursion to return the first number in the `nth` row. From the above pattern, you can observe the following:

- The first number of the 1st row is 1
- The first number of the nth row is the first number of the $(n-1)$th row plus $(n-1)$

```
def first(n):
    """Return the first number in the nth row
    >>> first(1) # first number in 1st row is 1
    1
    >>> first(4) # first number in 4th row is 7
    7
    """
    if n == 1:
        ______(a)______
    ______(b)______
```

Your task is to implement this recursive function to return the first number in any row `n`. You may only write one statement on each line. You may not use ; to place two statements on the same line. You may not use any functions or methods, built-in or otherwise, other than the recursive function. You may not modify the level of indentation of any line.

(a) (2 pt) Fill in blank (a).

```
return 1
OR
return n
```

(b) (3 pt) Fill in blank (b).

```
return n-1 + first(n-1)
```

Finally, you need to write a function `row_sum(n)` which takes an integer `n` representing the row number (assume $n \geq 1$). The function should return the sum of all numbers in the `nth` row. This function must call the function `first()` to determine the first number of the `nth` row.

Hint: Looking at the triangle of numbers on the previous page, and if you know the first number of a row, can you find a pattern that allows you to find the other numbers in that row?

```
def row_sum(n):
    """Return the sum of all numbers in the nth row
    >>> row_sum(1)
    1
    >>> row_sum(2)
    5
    >>> row_sum(3)
    15
    >>> row_sum(4)
    34
    """
    total = 0
    ______(c)______
        ______(d)______
    return total
```

Your task is to implement this function by calling `first()`. Answers that do not call the function `first()` will receive no credit. You may only write one statement on each line. You may not use ; to place two statements on the same line. You may not modify the level of indentation of any line.

(c) (2 pt) Which of these could fill in blank (c)?

- ◯ `for i in range(first(n)):`
- ◯ `for i in len(n):`
- ◯ `while i < n:`
- ◯ `for i in n:`
- ◯ `for i in range(n + 1):`
- ◯ `for i in first(n):`
- ◯ `for i in range(n - 1):`
- ◯ `for i in range(n, first(n)):`
- ◯ `if i < n:`
- ● `for i in range(n):`

(d) (3 pt) Fill in blank (d).

```
total = total + first(n) + i
OR
total += first(n) + i
```